API Backend

Build a Production-Ready REST API with Node.js

■ LEARNING OUTCOME

By the end of this module, you will build a full-featured REST API with Express, implement middleware for auth, validation, and error handling, structure code professionally, document with Swagger/OpenAPI, and test with Postman.

■ Skills You Will Gain
• Structure an Express project for production scale
• Implement middleware chain: logger, cors, auth, validation, error handler
• Write clean controller and service layer architecture
• Document API endpoints with Swagger/OpenAPI
• Test API with Postman collections and automated tests
• Handle async errors, rate limiting, and security headers

■ **A well-designed API is the backbone of every modern application. The same API can serve a React app, mobile app, and third-party integrations. Clean, documented, secure APIs define senior backend developers.**

SECTION 1

Project Structure and Express Setup

```
backend/ src/ routes/ # route definitions (HTTP verbs + paths) controllers/ # handle req/res, call services services/ # business logic (no req/res objects) models/ # database schemas and queries middleware/ # auth, validation, rate-limit, error utils/ # helpers, constants, formatters config/ # env vars, database connection app.js # Express app setup server.js # start server .env .env.example package.json // src/app.js const express = require('express'); const cors = require('cors'); const helmet = require('helmet'); const rateLimit = require('express-rate-limit'); const morgan = require('morgan'); require('express-async-errors'); // makes async errors auto-pass to error handler const app = express(); // SECURITY app.use(helmet()); // sets 14 security headers automatically app.use(cors({ origin: process.env.ALLOWED_ORIGINS?.split(',') || 'http://localhost:5173', methods: ['GET','POST','PUT','PATCH','DELETE'], allowedHeaders: ['Content-Type','Authorization'], })); // RATE LIMITING const limiter = rateLimit({ windowMs: 15 * 60 * 1000, // 15 minutes max: 100, // max 100 requests per window message: { error: 'Too many requests, please try again later.' }, }); app.use('/api/', limiter); // AUTH endpoint: stricter limit const authLimiter = rateLimit({ windowMs: 15*60*1000, max: 10 }); app.use('/api/auth/', authLimiter); // LOGGING app.use(morgan(process.env.NODE_ENV === 'production' ? 'combined' : 'dev')); // BODY PARSING app.use(express.json({ limit: '10kb' })); app.use(express.urlencoded({ extended: true, limit: '10kb' })); // ROUTES app.use('/api/auth', require('./routes/auth')); app.use('/api/users', require('./routes/users')); app.use('/api/tasks', require('./routes/tasks')); // 404 app.use((req, res) => res.status(404).json({ error: `Route ${req.path} not found` })); // GLOBAL ERROR HANDLER (must be last, must have 4 params) app.use(require('./middleware/errorHandler')); module.exports = app;
```

SECTION 2

Middleware Chain and Error Handling

```
// middleware/auth.js -- JWT verification const jwt = require('jsonwebtoken'); const User = require('../models/User'); exports.protect = async (req, res, next) => { const auth = req.headers.authorization; if (!auth?.startsWith('Bearer ')) { return res.status(401).json({ error: 'No token provided' }); } try { const payload = jwt.verify(auth.split(' ')[1], process.env.JWT_SECRET); req.user = await User.findById(payload.id).select('-password'); if (!req.user) return res.status(401).json({ error: 'User not found' }); next(); } catch (err) { if (err.name === 'TokenExpiredError') return res.status(401).json({ error: 'Token expired, please log in again' }); res.status(401).json({ error: 'Invalid token' }); } }; exports.restrict = (...roles) => (req, res, next) => { if (!roles.includes(req.user.role)) return res.status(403).json({ error: 'Insufficient permissions' }); next(); }; // middleware/errorHandler.js -- catches all thrown errors exports.errorHandler = (err, req, res, next) => { const status = err.statusCode || 500; if (process.env.NODE_ENV !== 'production') { console.error(err.stack); } res.status(status).json({ error: err.message || 'Internal server error', ...(process.env.NODE_ENV !== 'production' && { stack: err.stack }), }); }; // utils/AppError.js -- custom error class class AppError extends Error { constructor(message, statusCode) { super(message); this.statusCode = statusCode; } } module.exports = AppError; // Using AppError in controllers const AppError = require('../utils/AppError'); exports.getOne = async (req, res, next) => { const task = await Task.findById(req.params.id); if (!task) throw new AppError('Task not found', 404); if (task.userId !== req.user.id) throw new AppError('Not authorised', 403); res.json(task); };
```

SECTION 3

Service Layer and OpenAPI Documentation

```
// services/taskService.js -- pure business logic, no req/res const db = require('../config/db'); const AppError = require('../utils/AppError'); exports.getAllTasks = async ({ userId, status, priority, page=1, limit=20 }) => { const offset = (page-1) * limit; const [tasks, total] = await
```

```
Promise.all([ db.task.findMany({ where:{ userId, ...(status&&{status}), ...(priority&&{priority})
}, skip:offset, take:+limit, orderBy:{createdAt:'desc'} }), db.task.count({ where:{ userId } }),
]); return { tasks, total, page:+page, pages:Math.ceil(total/limit) }; }; exports.createTask =
async (userId, data) => { return db.task.create({ data: { ...data, userId } }); };
exports.updateTask = async (userId, taskId, data) => { const existing = await db.task.findUnique({
where:{ id:taskId } }); if (!existing) throw new AppError('Task not found', 404); if
(existing.userId !== userId) throw new AppError('Forbidden', 403); return db.task.update({ where:{
id:taskId }, data }); }; // Swagger/OpenAPI documentation -- install: npm install swagger-jsdoc
swagger-ui-express /** * @swagger * /api/tasks: * get: * summary: Get all tasks for the
authenticated user * tags: [Tasks] * security: * - bearerAuth: [] * parameters: * - in: query *
name: status * schema: { type: string, enum: [todo, in-progress, done] } * - in: query * name: page
* schema: { type: integer, default: 1 } * responses: * 200: * description: List of tasks * content:
* application/json: * schema: * type: object * properties: * tasks: { type: array, items: { $ref:
'#/components/schemas/Task' } } * total: { type: integer } * 401: * description: Unauthorized */
```

■ PRACTICE Q & A

Q1. What is the difference between a controller and a service layer?

A: *Controller: handles HTTP concerns (req, res, status codes, response format). Service: pure business logic with no knowledge of HTTP. Services are reusable (same logic from REST API, CLI, or background jobs) and easier to unit test.*

Q2. Why use helmet.js in an Express app?

A: *helmet() sets 14 HTTP security headers in one call: prevents clickjacking (X-Frame-Options), XSS (Content-Security-Policy), sniffing (X-Content-Type-Options), and more. Without it, your API is vulnerable to common attacks by default.*

Q3. What is rate limiting and why is it essential?

A: *Restricts how many requests a client can make in a time window. Without it: brute-force attacks on login (try 10,000 passwords), credential stuffing, DDoS, and resource exhaustion. Set strict limits on auth endpoints (10/15 min) and general limits (100/15 min).*

Q4. What does express-async-errors do?

A: *Monkey-patches Express so that async errors thrown inside async route handlers automatically pass to the next(err) error handling middleware. Without it, uncaught promise rejections in async functions crash the process silently.*

Q5. What is the four-parameter error handler in Express?

A: *Express recognises an error-handling middleware by its (err, req, res, next) signature -- the four parameters. It must be defined LAST after all routes. Any call to next(err) or throw inside async routes (with express-async-errors) reaches it.*

■ API Backend Cheat Sheet	
`app.use(helmet())`	Add 14 security headers
`app.use(cors({origin:...}))`	Configure CORS origins
`rateLimit({windowMs, max})`	Limit requests per window
`require('express-async-errors')`	Auto-catch async throws
`app.use((err,req,res,next)=>{}`	4-param global error handler
`class AppError extends Error`	Custom error with status code
`jwt.verify(token, secret)`	Verify JWT in auth middleware
`req.user = await User.find()`	Attach user to request
`restrict(...roles)`	Role-based access control
`morgan('combined')`	HTTP request logging
`service layer`	Pure business logic, no req/res
`@swagger JSDoc comments`	OpenAPI documentation